SIMATS ENGINEERING

SOFTWARE TESTING – SCENARIO-BASED ASSESSMENT

Student Name: PRADEEP.S

Registration Number: 192521024

Course Code:CSA3711

Course name: SOFTWARE TESTING

1. Online Banking Fund Transfer – ECP, BVA and Random Testing

The failures near minimum and maximum transfer limits indicate that the existing test suite may not adequately test boundary values. Testing only ordinary transfer amounts can miss off-by-one errors, incorrect comparisons, and limit-validation defects.

Equivalence Partitioning (ECP): Divide transfer amounts into valid and invalid classes. Assuming the permitted amount is L to U, the classes are: amount below L (invalid), L through U (valid), and amount above U (invalid). Also test zero, negative values, non-numeric input, blank input, insufficient balance, and a transfer exceeding the available balance where applicable.

Boundary Value Analysis (BVA): Test L−1, L, L+1, U−1, U, and U+1, using the smallest currency unit supported by the application. For example, if the allowed range is ₹100–₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. Verify the displayed message, account balance, transaction record, and whether funds are debited exactly once.

Random testing can reveal unexpected combinations, so it should be retained as supplementary fuzz/random testing. It should not replace systematic ECP and BVA because random values may rarely hit critical boundaries. The improved strategy is to derive tests from requirements, prioritize boundaries and invalid classes, add decision-table tests for balance/limit/beneficiary conditions, and use regression tests after every fix. Include duplicate requests, network timeout, and retry cases to verify that a transfer is not processed twice. This structured approach provides repeatable, high-value defect detection.

2. Healthcare Application – White-Box Coverage and Control Flow Graph

Critical post-deployment defects suggest that test adequacy was judged by a limited metric or that important decision outcomes and execution paths were not tested. Statement coverage only checks

whether each executable statement ran at least once; it does not guarantee that every decision outcome or important combination was exercised.

A conceptual control flow graph (CFG) can be represented as: Entry → Validate input → [Input valid?]. If No, show validation error → Exit. If Yes, → Check patient/record eligibility → [Eligible?]. If No, reject request → Exit. If Yes, → Process clinical rule → [Rule satisfied?]. If Yes, save result and audit log → Exit; if No, return appropriate warning and audit log → Exit. Each decision is a node with true and false outgoing edges.

Review the test suite against statement, branch, condition, and basis-path coverage. Statement coverage may miss false branches; branch coverage can still miss interactions among conditions; complete path coverage is generally impractical when loops or many decisions create numerous paths. Identify untested CFG edges and independent paths, especially invalid input, ineligible records, failed clinical rules, exceptions, and database failures.

The improved plan is to map requirements to tests, achieve 100% statement and branch coverage where feasible, add condition/MC/DC tests for safety-relevant Boolean logic, and use basis-path testing guided by cyclomatic complexity. Add boundary, negative, integration, and regression tests, with independent review of clinical rules. This increases confidence that important decisions and failure handling work correctly, reducing patient-safety and data-integrity risks.

3. E-Commerce Price, Quantity and Discount – Combined Input Testing

Testing price, quantity, and discount independently is insufficient because a defect may occur only when two or more values interact. The existing black-box strategy likely missed combinations such as a discount greater than the subtotal, zero quantity, or a high quantity combined with a price boundary.

First define the requirements. Example assumptions: price must be greater than zero, quantity must be a positive integer, and discount must be between 0% and the maximum allowed discount. Use ECP to create valid and invalid classes for each field. For price, test valid positive prices, zero, negative, blank, and non-numeric values. For quantity, test positive integers, zero, negative, fractions, and invalid text. For discount, test 0%, an ordinary allowed value, the maximum, a value above the maximum, and negative values.

Use BVA at the minimum and maximum permitted values for each field. Then use decision-table testing for important combinations: valid/invalid price, valid/invalid quantity, valid/invalid discount, customer eligibility, and expected result. For example, valid inputs should calculate subtotal = price × quantity

and apply the permitted discount; invalid inputs should be rejected without producing a negative total. Test rounding and currency precision.

Exhaustively testing every possible value is unrealistic. Use pairwise/combinatorial testing for broad interaction coverage, while retaining targeted tests for high-risk combinations and business rules. Add negative tests, expected-result checks, and regression tests. This provides more systematic coverage than isolated input testing while controlling the number of test cases.

4. Nested Conditions and Loops – Complete Path Testing Strategy

Partial path coverage in a module with nested conditions and loops indicates that the team may not have identified all decision outcomes or independent paths. A correct CFG should include every statement, decision, loop entry, loop exit, and exception/early-return edge. Missing edges lead to untested logic and may hide defects in combinations of nested conditions.

Construct the CFG from the source code and calculate cyclomatic complexity, typically $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components (normally 1 for a single module). The result estimates the number of linearly independent paths needed for basis-path testing; it does not mean that every possible loop iteration path must be enumerated.

Create a basis set of independent paths and ensure every decision outcome is exercised. For each loop, test zero iterations where permitted, one iteration, two iterations, typical iterations, maximum supported iterations, and termination conditions. For nested loops, test inner-loop and outer-loop boundaries and verify that loop counters and exit conditions change correctly. Add tests for break/continue, exceptions, and invalid data if present.

Achieving 100% branch coverage and basis-path coverage where feasible provides stronger evidence than statement coverage alone. Because loops can create an enormous or infinite number of paths, use risk-based path selection and explicit loop-boundary testing rather than claiming unrestricted complete path coverage. Review CFGs and coverage reports, automate regression tests, and prioritize safety- or business-critical paths.

5. Financial Application – Integrated Black-Box and White-Box Testing

Defects reaching production despite both black-box and white-box testing suggest that the methods may have been applied separately, coverage criteria may be weak, or tests may not reflect real requirements and failure conditions. High coverage alone does not prove that expected outputs are correct.

Black-box testing should derive cases from requirements without relying on internal code. Apply ECP to valid and invalid input groups and BVA to transaction limits, dates, balances, interest rates, and currency precision. Use decision tables for combinations such as account status, available balance, transaction type, limits, and authorization. Include invalid data, duplicate requests, timeouts, and recovery scenarios.

White-box testing should inspect the implementation. Build CFGs for critical modules, identify independent paths using cyclomatic complexity, and test both outcomes of decisions. Use condition coverage or MC/DC for complex financial rules, and review exception handling, loops, and transaction rollback paths. Measure statement, branch, condition, and basis-path coverage as appropriate, while recognizing that metrics do not replace requirement-based tests.

Integrate both approaches through traceability: each requirement maps to black-box tests, and each high-risk code decision maps to white-box tests. Run unit, integration, system, security, and regression tests in a controlled pipeline. Use code review, independent test-data checks, production-like environments, and defect root-cause analysis. This hybrid strategy improves input, logic, and interaction coverage and reduces the chance that an untested branch or misunderstood requirement reaches production.

6. HR Payroll Bonus – Exact Service-Year Boundaries

Incorrect bonuses at exactly 5, 10, and 15 years strongly suggest boundary or comparison-operator errors, such as using greater-than instead of greater-than-or-equal, or assigning an employee to the wrong service band. Exploratory testing may find such problems, but a systematic boundary-focused suite is more repeatable.

Assume the bonus bands are defined by the official payroll policy. For every threshold T = 5, 10, and 15 years, test just below T, exactly T, and just above T. If service is calculated in completed years, use dates that result in T−1 day, the exact anniversary date, and T+1 day; do not rely only on rounded year values. Verify the policy's treatment of leap years and the date on which service is calculated.

ECP should group employees into each service band, below the first threshold, and above the last threshold. Create separate classes for each performance rating, including valid ratings and invalid/missing ratings. Use a decision table crossing service band with performance rating and expected bonus percentage/amount. Check rounding, salary basis, eligibility, and the final payout.

Exploratory testing is useful for discovering unexpected behavior but may not consistently hit exact thresholds or cover all rating combinations. ECP and BVA are superior for this known banded financial

logic because they are requirement-driven, repeatable, and easy to automate. Confirm expected results against the approved payroll policy and include regression tests for all three thresholds.

7. Laboratory Temperature Control – CFG and MC/DC

The missed alarm during a sensor-malfunction sequence suggests that a condition or branch combination was not adequately tested. Because this is a safety-related controller, tests must verify both normal control and safe responses to invalid sensor readings.

A conceptual CFG is: Start → Read temperature and sensor status → [Sensor valid?]. If No, enter safe state, disable unsafe control actions, activate alarm, and log fault → End/monitor again. If Yes, → [Temperature above upper limit?]. If Yes, activate cooler and check alarm condition; if No, → [Temperature below lower limit?]. If Yes, activate heater; otherwise keep normal control. At every relevant stage, check whether an alarm is required by the specification. The exact CFG must be updated to match the actual implementation.

Branch testing must exercise true and false outcomes of every decision, including valid/invalid sensor status, high/normal/low temperature, and alarm activation. For MC/DC, show that each atomic condition can independently change the outcome of its decision while other conditions are held fixed. For example, if Alarm = SensorFault OR (TemperatureHigh AND CoolingFailure), design pairs that independently demonstrate the effect of SensorFault, TemperatureHigh, and CoolingFailure. The precise test pairs depend on the implemented Boolean expression.

Aim for 100% MC/DC on safety-critical decisions, plus branch coverage, boundary temperatures, sensor disconnection/out-of-range values, stale readings, and fault-recovery tests. Use hardware-in-the-loop tests where possible and verify alarms, logs, and safe-state behavior. MC/DC improves confidence in decision logic but does not by itself prove the whole system safe; requirements review, integration testing, and hazard analysis are also necessary.

8. Travel Booking Promo Codes – Decision Table Testing

Testing each promo code in isolation misses interactions among codes, price, and user tier. Two individually valid promotions may be incompatible when combined, or discounts may exceed the booking price and produce a negative total.

Define the rules first: booking price must be positive; each promo code must be valid and unexpired; stacking rules must specify whether codes can be combined; user-tier eligibility must be checked; and the total must never be below zero. These are assumptions to confirm with the actual business policy.

Build a decision table with conditions such as valid price, promo A valid, promo B valid, codes stackable, user tier eligible, and discount total not exceeding the price. Actions include accept both codes, apply only one code, reject a code with a message, or calculate the final total. Test representative rules: no code, one valid code, two compatible codes, two incompatible codes, expired code, tier-ineligible code, discount equal to price, and discounts whose sum exceeds price. Also test zero/negative price and malformed codes.

Use cause-effect graphing if rules are numerous and dependent, then simplify the resulting table by merging rules with identical outcomes. Pairwise testing can cover additional interactions, but high-risk stacking and negative-total combinations must remain explicit. Assert that total = max(0, price − permitted discounts) only if this matches the approved policy; otherwise assert the specified rejection or discount-priority behavior. This approach improves interaction coverage without testing every theoretical combination.

9. Database Migration – Infinite Loops, Independent Paths and Complexity

High statement coverage does not guarantee correct loop behavior. A statement may execute once while loop termination, boundary values, nested-loop interactions, or exceptional records remain untested. Infinite-loop crashes often result from a counter not being updated, an incorrect exit condition, a record that repeatedly fails validation, or an error path that returns to the loop without progress.

Construct a CFG for the nested record-validation loops, including loop entry, body, increment/update, exit, error handling, and retry edges. Calculate cyclomatic complexity $V(G) = E - N + 2P$. For a single connected component, it is $E - N + 2$. Use this to identify a basis set of linearly independent paths, then design tests that exercise each independent decision outcome.

For every loop, test zero records, one record, two records, typical volume, maximum supported volume, and termination after the final record. For nested loops, include empty inner collections, one and multiple inner records, malformed records, duplicate records, and errors during processing. Verify that every iteration makes progress and that retry limits or timeouts prevent endless execution. Use large-data and stress tests to detect performance-related hangs.

Statement coverage should be supplemented with branch coverage, basis-path tests, loop-boundary tests, and assertions on termination. Instrument loop counters or timeouts in test environments and compare migrated record counts and checksums with the source database. Cyclomatic complexity helps estimate independent paths, but it does not enumerate every possible loop sequence. These measures improve termination reliability and migration correctness.

10. Food Delivery Tracker – State Transition Testing

The tracker getting stuck when a driver cancels mid-delivery indicates that the cancellation transition or its recovery state may be missing. ECP and BVA mainly classify input values; they do not adequately test sequences of events and state-dependent behavior.

Define states such as Ordered, Preparing, Out for Delivery, Delivered, Cancelled, and Reassignment Pending. Define permitted transitions from the business requirements. For example: Ordered → Preparing; Preparing → Out for Delivery; Out for Delivery → Delivered; and Out for Delivery → Reassignment Pending when the driver cancels. Reassignment Pending may lead to Out for Delivery with a new driver or to Cancelled/refund processing according to policy. Delivered should normally be a terminal state and should not revert to Preparing.

Test each valid transition and important invalid transition. Scenarios include driver cancellation before pickup, cancellation after pickup, no replacement driver, successful reassignment, customer cancellation, duplicate status events, delayed network updates, and app restart while a transition is pending. Verify status display, notifications, refund handling, timestamps, and that the order does not remain indefinitely in a transient state.

Use transition coverage and transition-pair coverage, plus sequence tests for the cancellation flow. Check state persistence and synchronization between server and mobile app. This technique finds missing or invalid event sequences that isolated input classes may not reveal, improving workflow consistency and customer trust.

11. Cybersecurity Authentication – Hybrid Testing and Error Guessing

Brute-force vulnerabilities indicate that testing may have focused on successful login and basic invalid credentials without adequately checking repeated attempts, timing, rate limits, or interactions among authentication conditions. Testing should be authorized and performed in a controlled environment.

Black-box tests should verify password rules, OTP validity and expiry, maximum attempts, account lockout or throttling, recovery behavior, and responses for unfamiliar IP locations. Use error guessing to target likely failures: repeated wrong passwords, repeated OTP submissions, OTP reuse, expired OTP, simultaneous login attempts, reset-flow abuse, and differences in error messages that could reveal account existence. Check that limits cannot be bypassed simply by restarting a session or changing a client-side value.

White-box testing should construct a CFG for password validation, OTP generation/verification, attempt counting, IP-risk evaluation, and allow/deny decisions. Use path analysis to test independent paths such

as valid password + valid OTP + acceptable risk, invalid password, expired OTP, attempt limit exceeded, and suspicious location. Review whether all denial paths update counters and whether errors fail securely.

Combine tests with security requirements, code review, dependency checks, logging review, and regression tests. Verify server-side enforcement, secure OTP randomness and storage, rate limiting, monitoring, and safe recovery. Avoid relying on a single coverage percentage: path coverage checks control flow, while adversarial black-box cases check externally observable weaknesses. This hybrid approach improves resistance to abuse and helps ensure security controls work together.

12. E-Learning Subscription Upgrade – Date-Based ECP

Errors on the exact expiration date suggest ambiguity about whether a subscription remains active through the expiration day or expires at its start. The expected behavior must be defined precisely, including time zone, timestamp precision, and whether the expiry value is inclusive or exclusive.

Apply ECP to date ranges: before the start date, within the active period, exactly at the expiration boundary, after expiration, and invalid or missing dates. Apply ECP to subscription types: Free, Basic, and Premium, and to course types that are included or excluded for each tier. Create a decision table mapping current tier, requested tier, course eligibility, payment success, and upgrade time to the expected access result.

For BVA, test immediately before, exactly at, and immediately after expiration—for example, one smallest supported time unit on either side of the boundary. Test upgrades before expiry, on the expiry date, after expiry, and during payment delay. Verify access rights, billing, renewal date, course progress, and whether an upgrade is applied once only.

Include invalid transitions, failed payments, duplicate requests, and time-zone differences. ECP gives representative coverage of the defined date and subscription classes, while BVA targets the exact boundary where the observed failure occurs. Requirement traceability and automated regression tests make the result repeatable and help ensure users move between tiers without unexpected loss or excess access.

13. Smart-Home Lighting – Branch and Condition Coverage

Statement coverage alone is inadequate because executing a statement does not prove that every switch case or Boolean outcome has been tested. A lighting algorithm may behave incorrectly for a particular sensor range, mode, or combination of conditions even when all lines have executed.

Review the source and draw a CFG containing the sensor read, validation, switch-case branches, Boolean decisions, dimming calculation, and hardware output. For each switch statement, test every defined case and the default case. For every decision, test true and false outcomes. Condition coverage requires each atomic Boolean condition to evaluate both true and false at least once; branch coverage requires each decision outcome to be taken.

For example, if dimming depends on SensorValid AND (AmbientLightLow OR ManualOverride), design tests that independently vary SensorValid, AmbientLightLow, and ManualOverride. Test sensor readings at each threshold and just below/above it, invalid readings, missing sensor data, and conflicting manual/automatic commands. Confirm that output levels are within supported limits and that the physical light responds correctly.

Combine statement, branch, and condition coverage with boundary tests and hardware-in-the-loop integration tests. Add MC/DC if the Boolean logic is safety-critical or the requirements demand it. Review coverage reports for missing cases and automate regression tests. This approach can reveal logical gaps and interface mismatches that statement coverage alone would miss.

14. Logistics Shipping – Robust Boundary Value Analysis

The failure to test zero and negative weights shows that the input-validation suite did not adequately cover invalid boundaries. If shipping weight must be positive, zero and negative values must be rejected before the cost calculation; otherwise the system may produce incorrect prices or accept invalid shipments.

Assume the minimum permitted weight is greater than zero and the maximum supported weight is Wmax. Robust BVA tests values below, at, and above the lower and upper limits. For a minimum of 0.01 kg, representative tests are −0.01 kg, 0 kg, 0.01 kg, 0.02 kg, Wmax−0.01 kg, Wmax, and Wmax+0.01 kg, adjusted to the system's supported precision. Also test blank, alphabetic, extremely large, and excessive-decimal inputs.

Test valid and invalid destination zones and delivery speeds, as shipping cost depends on all three fields. Use decision tables for combinations of weight class, destination zone, and delivery speed. Verify that invalid weight is rejected with a clear message, no shipment is created, and no charge is applied. Check unit conversion, rounding, and overflow handling.

Negative testing is necessary because real users, integrations, or corrupted data may supply invalid values. Robust BVA specifically targets boundary and just-outside-boundary defects. Combine it with

ECP, input validation, and regression tests to improve reliability and prevent incorrect charges or invalid shipment records.

15. Medical Imaging – Prime Data Sizes and Basis Path Testing

Inconsistent results for prime-number data sizes suggest that array partitioning, parallel chunk allocation, remainder handling, synchronization, or loop bounds may be incorrect. Random testing alone is inefficient for this pattern because it may not select prime sizes or exercise the specific branches responsible for remainder data.

Inspect the source code and build a CFG for data validation, size calculation, partitioning, parallel worker loops, remainder processing, synchronization, and result aggregation. Calculate cyclomatic complexity $V(G) = E - N + 2P$ and identify a basis set of independent paths. Test each independent path with carefully selected data sizes rather than relying only on random values.

Use sizes that exercise different partition cases: zero if allowed, one element, values smaller than the worker count, exact multiples of the chunk size, one less and one more than a multiple, and prime sizes both smaller and larger than the chunk size. Include uneven partitions, empty chunks, maximum supported datasets, and invalid input. Compare parallel output with a trusted sequential reference implementation and verify that each element is processed exactly once.

Add stress, race-condition, repeatability, and synchronization tests, along with branch coverage and regression tests. Basis-path testing targets independent control-flow logic; data-size partitions target arithmetic and loop boundaries. Together they improve mathematical correctness and system stability. Random testing may remain as supplementary fuzzing, but it should not replace structured white-box tests.

Quick Reference: Key Testing Concepts

- **Equivalence Class Partitioning (ECP):** Divides input data into groups expected to behave similarly; select representative values from valid and invalid groups.
- **Boundary Value Analysis (BVA):** Tests values at and around the minimum and maximum limits, where defects commonly occur.
- **Decision Table Testing:** Checks combinations of conditions and the actions expected for each rule.
- **Control Flow Graph (CFG):** Represents statements, decisions, and control transfers in a program.
- **Statement Coverage:** Measures whether executable statements ran at least once.
- **Branch Coverage:** Measures whether each decision outcome was exercised.

- **Condition Coverage:** Checks that each atomic Boolean condition evaluates both true and false.
- **MC/DC:** Shows that each atomic condition can independently affect the outcome of a decision.
- **Basis Path Testing:** Tests a set of linearly independent paths, often guided by cyclomatic complexity.
- **State Transition Testing:** Checks valid and invalid transitions between system states in response to events.

Submission Checklist

- Add student name and registration number on the first page.
- Confirm all assumed limits and business rules against the lecturer's or system's stated requirements.
- Review each answer and ensure you understand the reasoning before submission.
- Complete and sign the original-work declaration required by the assessment.